Università degli Studi di Modena e Reggio Emilia

Facoltà di Ingegneria – Reggio Emilia

Verso la Programmazione ad Oggetti

Prof. Franco Zambonelli

Corso di Reti di Calcolatori
Anno Accademico 2005/2006

VERSO UNA TECNOLOGIA OBJECT-ORIENTED

MOTIVAZIONI

- progettare sistemi software di grandi dimensioni richiede *adeguati supporti*
- "crisi del software": i costi di gestione diventano *preponderanti* su quelli di produzione
- il software dovrebbe essere **protetto, riusabile, documentato, modulare, incrementalmente estendibile**

IL PUNTO CHIAVE

- i linguaggi di programmazione devono fornire
  - *non solo* un modo per esprimere *computazioni*
  - ma anche un modo per **dare struttura** alla descrizione
  - e un supporto per **organizzare bene il processo produttivo del software**

L'OBIETTIVO

- costruzione ***modulare* e *incrementale*** del software
- ottenuta per **estensione** / **specializzazione** di ***componenti***
  ➔ tecnologia ***component-based***
- Le strutture dati e le strutture di controllo (programmazione strutturata) *non bastano*
- Le funzioni e le procedure *non bastano*

LE NECESSITA'

- servono strumenti per la costruzione ***modulare* e *incrementale*** del software

CRISI DIMENSIONALE

Programmi di piccole dimensioni

- accento sull'algoritmo
- diagrammi di flusso
- programmazione strutturata

Non appena il programma cresce in dimensioni, non si riesce più a gestirlo: bisogna diminuire la complessità!

Programmi di Medie Dimensioni

- funzioni e procedure come astrazioni di istruzioni complesse
- decomposizione degli algoritmi in blocchi funzionali (anche su file diversi, per permettere lo sviluppo di equipe)

Alla base rimane il concetto di "algoritmo": tutto il programma è un grosso algoritmo spezzato su più funzioni/file

Programmi di Grandi Dimensioni

- *tipicamente trattano grandi moli di dati **MA***
  - la decomposizione funzionale non è adeguata
  - non c'è accoppiamento dati-funzioni che li elaborano: dati elaborati globalmente da tutto il programma
- *tipicamente devono essere sviluppati da team **MA***
  - la decomposizione funzionale e il disaccoppiamento dati-funzioni non permette la decomposizione del lavoro
- *tipicamente trattano ed elaborano dati relativi ad entità del mondo reale (persone, oggetti, grafici, documenti), e interagiscono con entità del modo reale MA*
  - le entità del mondo reale non sono "dati" su cui operano delle funzioni, ma sono entità con attributi e comportamenti, e devono essere trattate in modo coerente alla loro essenza

CRISI GESTIONALE

Il costo maggiore nel processo di produzione del software è dovuto al suo mantenimento:

- correttivo
- adattativo

Programmi di piccole dimensioni

- non difficilissimo trovare gli errori
- propagazione degli effetti delle modifiche limitate intrinsecamente dalle dimensioni del programma

Programmi di Medie Dimensioni

- gestione basata sulle procedure per l'individuazione degli errori
- gli effetti delle modifiche non sono limitati alle procedure in cui tale modifiche sono fatte, ma si propagano, a causa del non-accoppiamento dati funzioni

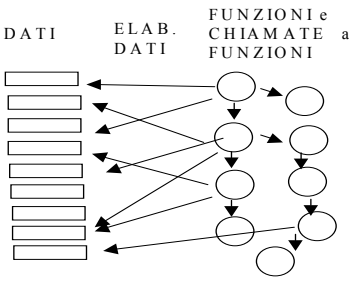
Programmi di Grandi Dimensioni

- quasi impossibile trovare gli errori
  - se le modifiche si possono propagare non è possibile fare delle modifiche senza coinvolgere tutto il team di sviluppo

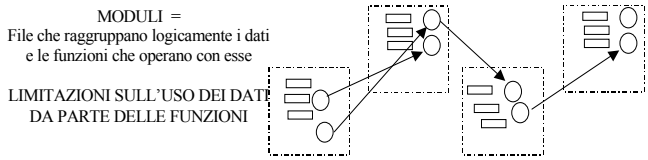
è necessario cambiare RADICALMENTE il modo di concepire, progettare e programmare il software!

IL CAMBIO DI PARADIGMA

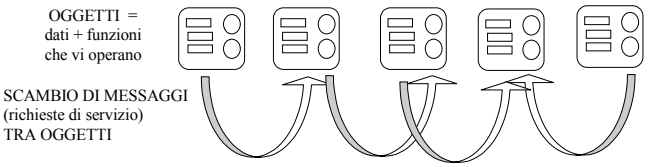
Tradizionale:



A Moduli:



Ad Oggetti:



CONCETTI CHIAVE

- Astrazione di Dato e Tipi di dato astratto (ADT)
- Modello Cliente / Servitore (*Client* / *Server*)
- Oggetti e Classi
- Moduli

ASTRAZIONE DI DATO

- Si focalizzare sulle *categorie concettuali* delle applicazioni
- Si costruisce un programma in termini di "entità", con propri attributi e comportamenti:
  - *attributi*: una struttura dati interna NASCOSTA (o non facilmente accessibile)
  - *comportamenti*: operazioni che possono AGIRE SUI DATI, che interagiscono tra loro

VANTAGGI

- agevola la modularità accoppiando dati e funzioni che su essi operano (diminuzione di complessità, certi dati è corretto che passono essere trattati solo tramite certe funzioni e viceversa).
- favorisce il controllo sull'integrità dei dati (solo certe funzioni possono agire sui dati e non funzioni arbitrarie che potrebbero fare dei danni o cose errate!). Se un dato risulta errato, si localizza facilmente in quale operazione è l'errore
- favorisce la creazione di componenti autonomi e "validati"
- minimizza la distanza concettuale tra problema e sua risoluzione

TIPO DI DATO ASTRATTO

Entità simili possono essere considerate "dello stesso tipo" (o della stessa classe), anche se con attributi specifici diversi.

ASTRAZIONE DI DATO: DEFINIZIONE

DATO CHE DEFINISCE UNA "ENTITA'" ASTRATTA (p.e. COME VIRTUALIZZAZIONE SOFTWARE DI UNA ENTITA' DEL MONDO REALE:

- i dati (o "attributi") che rappresentano tale entità – tipicamente non accessibili dall'esterno ma solo attraverso
- le operazioni (o "comportamenti" o "metodi") che su tali entità si possono effettuare

TIPO DI DATO ASTRATTO (ADT): tipo da cui è possibile definire ("**istanziare**") variabili come astrazioni di dato.

ESEMPI:

CONTATORE. Una entità che virtualizza qualcosa o qualcuno il cui compito è contare degli eventi

- Attributo numerico indica gli eventi che ha contato fino a quel momento
- Metodi per incrementare e per sapere fino a quanto ha contato fino a quel momento

STUDENTE. Una entità che rappresenta in software uno studente universitario

- attributi quali nome, cognome, numero di matricola, anno di corso, numero di esami dati, etc. etc.
- metodi per aggiornare l'anno di corso e gli esami dati

PROBLEMA: I linguaggi di programmazione tradizionali non permettono la definizione di astrazioni di dato e di tipi di dato astratto

ESEMPIO: il contatore

Approccio classico:

```
main()
{
…
int cont;

cont++;
cont--;

if (cont == MAX) {….}

cont = cont*cont;
/* puó avere senso per un intero ma non ha alcun senso
per una entità contatore */
}
```

PROBLEMI

- dov'è l'entità, il tipo di dato astratto, contatore?
- dove sono le operazioni ammesse sui dati di tipo contatore?
- possiamo fare cose NON SENSATE sul contatore
- stiamo usando le mosse elementari della macchina C (integer e operatori di incremento), non stiamo usando delle categorie concettuali di tipo contatore! non possiamo ri-usare il concetto di contatore!

ESEMPIO: studenti

Approccio Classico:

```
typedef struct Studente {
        char nome[20];
        char cognome[20];
        int matricola;
        int num_esami_dati;
        struct esami[29] {char nome[20]; int voto;}
                    }

void nuovo_esame_dato(Studente s, char *nome_esame, int
voto_preso)
{
   strcpy(s.esami[s.num_esami_dati].nome, nome_esame);
   strcpy(s.esami[s.num_esami_dati++].voto, voto_preso);
}
/* e altre funzioni varie…..*/

main()
{
Studenti s1, s2;


s1.nome = "pippo"; s2.cognome = "rossi";
/* NON BELLO: COSI' ACCEDO AI DATI INTERNI DI STUDENTE*/

/* POI POSSO FARE */
nuovo_esame_dato(s1, "Sistemi Operativi", 18);

/* MA ANCHE */
s1.esami[4].voto++; /* NON DOVREI! NON HA SENSO! Non e'
una operazione ammissibile sul tipo di dato*/

                            }
```

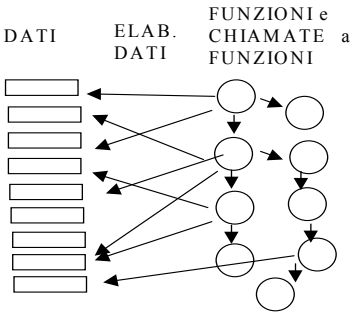

COME FARE:

- Cercare di costruire "astrazioni di dato" nei linguaggi tradizionali
  - Scomposizione di un programma su piu' file
  - Associare all'interno di uno stesso file i dati e le funzioni che operano su essi
  - Limitare l'uso dei dati in un file da parte di altri funzioni in altri file che potrebbero usarli in modo non corretto

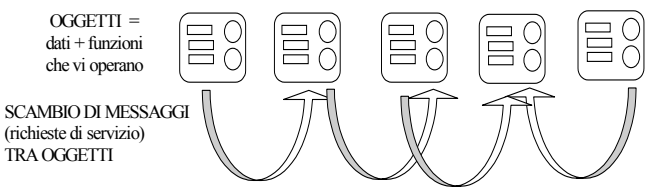
- Oppure:
  - Usare un nuovo linguaggio di programmazione, basato su principi diversi e dove:
  - PER DEFINIZIONE, i dati sono accoppiati alle funzioni a formare astrazioni di dato
  - SI PERDA il dualimo dati-funzioni → OGGETTI

IL CAMBIO DI PARADIGMA

Tradizionale:



Ad Oggetti:



ESEMPIO: il contatore

TIPO DI DATO ASTRATTO "Contatore"

DATI (Attributi che caratterizzano un contatore):

```
int valore_cont;
```

OPERAZIONI (quelle permesse su un tipo di dato "contatore")

```
void inc(){ valore_cont++;}
void dec(){ valore_cont--;};
int getValue(){ return valore_cont;};
```

Vorremmo Poter Fare:

```
main()
{Contatore cont; /* definisco un TDA Contatore*/
/* NEL LINGUAGGIO A OGGETTI SI DICE "ISTANZIARE" */

Contatore cont2; /* ne istanzio un'altro */

cont.inc(); /* invoco delle operazioni sul contatore */
cont.dec();

cont2.dec() /* le stesse operazioni le posso invocare
chiedendole all'altro contatore: sono dello stesso tipo,
ma ovviamente avranno attributi specifici diversi!*/

if(cont.getValue() > MAX) {….}

cont++; /* IL COMPILATORE DA' ERRORE: NON E' UN INTERO */

cont.valore_cont=cont*345;
cont.valore_cont++   /*   ERRATE   ENTRAMBE:   il   dato
valore_cont è accessibile solo tramite le operazioni di
cont */
}
```

In altre parole: vorremmo un linguaggio che spinga a pensare in termini di ADT

ESEMPIO: studenti

DATI:
```
char nome[20];
char cognome[20];
int matricola;
Cont num_esami_dati;
struct esami sostenuti {char * nome; int voto};
```

OPERAZIONI (alcune pensabili tra quelle ammissibili)

```
void nuovo_esame(char *nome_esame, int voto_preso);
Int calcola_media(void);
void iscrivi_anno_successivo();
```

Vorremmo Poter Fare:

```
main()
{
Studenti s1("Franco Zambonelli"), s2("Donald", "Duck");

s1.nuovo_esame("Sistemi Operativi", 28);
}
```

GROSSO CAMBIAMENTO DI PARADIGMA:

NON si richiede a funzioni di operare su dati

```
nuovo_esame(s1, "Sistemi Operativi", 28);
```

MA si richiede alle entità di eseguire delle funzioni

```
s1.nuovo_esame(s1, "Sistemi Operativi", 28);
```


VERSO I LINGUAGGI AD OGGETTI

Un linguaggio che permette di trattare tutti i dati in termini di astrazioni di dato, appartenenti a un tipo di dato astratto, è un linguaggio ad oggetti.

In pratica:
- si definiscono i tipi di dato su cui il nostro programma dovrà agire
- non si definiscono algoritmi nel senso proprio, ma si definiscono i "metodi" dei tipi di dato, come le operazioni che si possono effettuare su questi dati
- si creano "istanze" dei tipi di dato come se fossero variabili

In pratica:
- i tipi di dato sono descrivono "classi" di entità
- il programma farà uso di istanze di quelle classi, che sono chiamate "oggetti".

Ulteriori caratteristiche importanti di un linguaggio ad oggetti:
- si possono definire classi in termini di altre classi (p.e., la classe degli "studenti" si può definire in termini della classe "persone") **CONCETTO DI EREDITARIETA'**, lo vedremo più avanti
- non si invocano più "funzioni" ma si va a chiedere agli oggetti di eseguire delle operazioni, o dei "servizi". **MODELLO CLIENTE-SERVITORE**

- Dato un problema, basarsi su un insieme di *centri di servizio,* ognuno capace di eseguire un ben preciso *insieme di attività*

MODELLO CLIENTE/SERVITORE

- Dato un problema, basarsi su un insieme di *centri di servizio,* ognuno capace di eseguire un ben preciso *insieme di attività*
- Gli oggetti possono considerarsi come centri di servizio

Spostamento del fuoco
- NON chiamare una funzione che elabori dei dati MA
- chiedere a una entità software di svolgere un servizio

Esempio:
dato un vettore, NON invocare una procedura di ordinamento, MA chiedere a una entitá ordinatrice di farci il servzio di ordinarci il vettore

Esempio:
dato un file, non invocare una funzione che scriva dati sul file, ma chiedere al file stesso di svolgere il servizio relativo alla scrittura di file su di esso

- I clienti *non conoscono* l'organizzazione interna dei centri di servizio, e *non possono* accedere *direttamente* a essa
  - non interessa l'algoritmo (come sono fatte le cose) ma interessa che le cose vengano fatte
  - i clienti sono indipendenti da come sono fatti i servitori: facilita la manutenzione!

Nell'esempio precedente:

- il main chiede servizi allo Studente s1, il quale chiede servizi a un Contatore cont;

COSA VORREMO?

UN LINGUAGGIO PER DEFINIRE E ISTANZIARE ADT (oggetti), CHE POSSONO INTERAGIRE TRA LORO SOLO TRAMITE RICHIESTE DI SERVIZIO

```
Class Contatore {
    int valore_cont;  /* struttura dati interna */
    /* operazioni ammesse */
    void inc() {valore_cont++;}
    void dec() {valore_cont--;}
    int getValue() {return valore_cont;} }
Class Studente {
    /* struttura dati interna */
    char nome[20];
    char cognome[20];
    int matricola;
    Contatore num_esami_dati;
    struct esami[29] {char nome[20]; int voto;}
    /* operazioni ammesse */
    void nuovo_esame(char *nome_esame, int voto_preso)
           { int cont = num_esame_dati.getValue();
             esami[cont].nome = nome; …}
    …   }

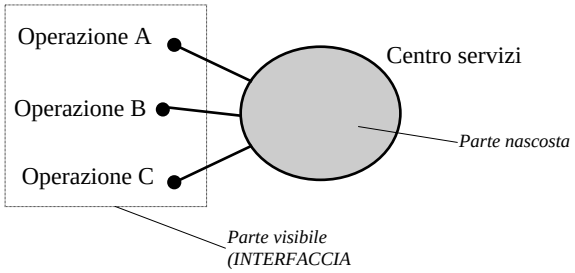
main()
{Studente s1;

s1.nuovo_esame("Fondamenti II", 30);
/* corretta */
nuovo_esame(Fondamenti II", 18);
/*   ERRATO:   a   quale   entità   stiamo   richiedendo
l'operazione?*/
s1.matricola++;
/* ERRATO non possiamo agire sulla struttura dati interna
se non tramite le operazioni ammesse */
}
```

MA QUESTO SAREBBE ESATTAMENTE UN LINGUAGGIO DI PROGRAMMAZIONE AD OGGETTI!

IL CONCETTO DI OGGETTO

- Un oggetto come ***astrazione di dato*** o ***centro di servizi***
- con una *parte visibile* → *INTERFACCIA*
- e una *parte nascosta*



LE OPERAZIONI DI INTERFACCIA RAPPRESENTANO:

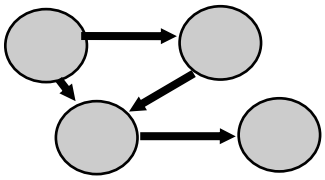
- I servizi che gli altri oggetti possono richiedere all'oggetto
- Modalità di trattamento dei dati dell'oggetto o di accesso ai dati

LA PARTE NASCOSTA CONTIENE

- **DATI**
  - Gli attributi che caratterizzano l'oggetto
  - Lo stato dell'oggetto
  - Sono accessibili solo attraverso richieste di servizio
- **OPERAZIONI PRIVATE**
  - Sfruttate dalle operazioni pubbliche ma non servizi esterni

ARCHITETTURA DI UN SISTEMA A OGGETTI

- Un ***insieme di oggetti***
- che **intergiscono gli uni con gli altri**
- senza conoscere nulla delle rispettive rappresentazioni concrete



DESCRIZIONE DI UN SISTEMA A OGGETTI

- Ogni oggetto appartiene a ("è istanza di") una data ***classe***
- **La classe *racchiude e incapsula la specifica*** di
  - **struttura** dell'oggetto (dati)
  - **comportamento** dell'oggetto (operazioni o metodi)
- Le classi ***integrano ed estendono il concetto di tipo e di* ADT**, rendendolo la base del linguaggio
- e possono essere correlate tra loro → *tassonomie* di ***ereditarietà*** che permettono il collegamento tra classi simil per raggruppare le proprietà simili e, in ultima analisi, ri-usare facilemente il codice comune alle classi correlate e ri-definire velocemente nuove classi

ESEMPIO: GESTIONE FACOLTA' UNIVERSITARIA

Descrizione problema: fare un programma che gestisca:
- i dati relativi ai corsi di laurea (iscritti, statistiche studenti, etc…)
- i dati relativi agli studenti (iscrizione studenti, esami superati, etc…)

Entità di tale mondo da rappresentare nel software:
- CdL
- Studenti
- Esami

Inoltre, serviranno
- entità "contatori" (p.e., per contare iscritti, numero di esami dati…etc.)
- entità "date" (p.e., date esami, date iscrizioni, etc…)

Quindi:

il nostro sistema software farà uso di diverse CLASSI ognuna delle quali descrive le proprietà generali di specifiche entità del mondo.

SEGUE ESEMPIO

Esempi di classi:

CLASSE Contatore:
descrive le proprietà degli oggetti contatore, ognuno dei quali ha:
- attributi: un valore intero che rappresenta il numero di volte che un certo evento si è verificato
- metodi: per incrementare il valore, metterlo a zero, controllarlo. Altre cose non si devono poter permettere.

Classe ESAMI
Descrive le proprietà degli esami dati dagli studenti:
- attributi: nome corso, promosso/bocciato, data superamento, voto
- metodi: per dare un voto positivo e leggere il nome dell'esame e il voto preso. NON SI DEVONO POTER FARE: modifica voto, modifica da promosso a bocciato.

CLASSE Studente
Descrive le proprietà generali degli oggetti studenti:
- attributi: nome, cognome, matricola, data iscrizione, anno iscrizione, elenco esami superati, numero totali esami superati
- metodi: aggiornare esami superati e numero esami, e anno di iscrizione, sapere se lo studente ha superato un certo esame. NON DI DEVONO POTER FARE: diminuire anno iscrizione o numero esami superati, cambiare nome/cognome.

NOTA: numero esami superati e anno di iscrizione sono attributi della classe dei contatori! Oggetti di una certa classe possono essere attributi di altre classi…

A partire dalle CLASSI si creano (istanziano) gli oggetti concreti che vanno a realizzare la architettura del sistema…. (come se si andassero a dichiarare variabili di quel tipo….)

SEGUE ESEMPIO:

Oggetti

Un oggetto speciale ha la funzione di "main", programma principale. A seconda delle specifiche caratteristiche della Facoltà, andrà a creare specifici oggetti.

Per la Facoltà di Ingegneria di Reggio Emilia:

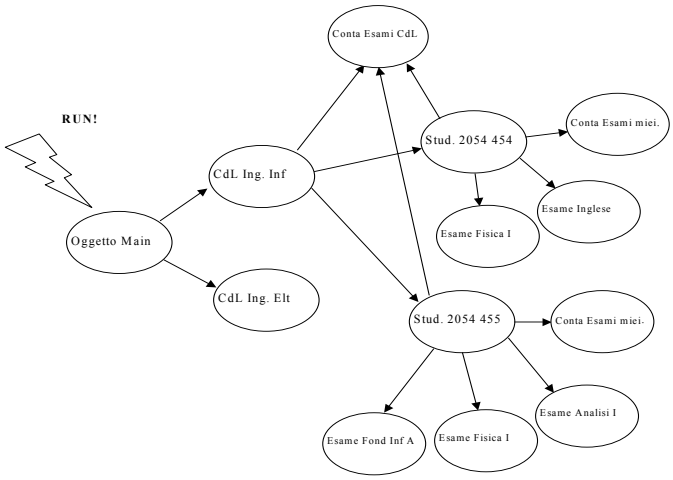
3 Oggetti CdL, con diversi attributi specifici:
- nomi diversi (ing. Gestionale, ing, E-business, etc…)
- tipo e numero di esami diversi
- studenti iscritti diversi….
- Inoltre, ogni oggetto della classe CdL ha come attributo un oggetto contatore che conta il numero totale di esami superati globalmente da tutti gli studenti

Per ogni nuovo studente che si iscrive:
- si crea un nuovo oggetto studente, che avrà gli attributi specifici della persona che si à icritta, e sarà poi inserito come attributo all'interno dell'oggetto CdL a cui si è iscritto

Per ogni nuovo studente, inoltre:
- si ha un nuovo oggetto data che rappresenta la data di iscrizione di quello studente
- si ha un nuovo oggetto contatore che conta il numero di esami superati da quello studente

SEGUE ESEMPIO:

Architettura



LEGENDA:
- ogni cerchio è un oggetto.
- le frecce rappresentano "relazioni" tra gli oggetti.

Quando si fa il run:
- il main crea (istanzia) gli oggetti CdL, inizializzandone gli attributi appositamente per ogni oggetto
- gli oggetti CdL creano i loro oggetti Contatori per contare gli esami generali.

Inizialmente, i CdL non hanno studenti iscritti e quindi il numero totale di oggetti studenti è zero e zero è anche il numero globale di esami superati in ogni CdL

SEGUE ESEMPIO:

Spiegazione Architettura e Funzionamento

Ad ogni nuova iscrizione di studente (l'input dei dati può essere fatto dal main, per esempio):
- il CdL deve creare un nuovo oggetto studente, e inizializzarne gli attributi come da input
- ogni oggetto studente creerà il proprio oggetto conta esami.

Ogni volta che uno studente supera un esame:
si crea un nuovo oggetto esame e ne si inizializzano i dati
- tale oggetto diventa un nuovo attributo dell'oggetto studente
- si incrementa il contatore esami di quello studente (meglio: l'oggetto studente chiede al suo oggetto contatore di eseguire il servizio di incremento)
- si incrementa il contatore esami globale del CdL (meglio: l'oggetto studente chiede all'oggetto contatore esami del CdL di eseguire il servizio di incremento).

Quando, a fini statistici, si vuole sapere il numero di studenti che ahnno superato Analisi I:
- si chiede all'oggetto CdL di realizzare tale servizio (ammesso che sia tra i medoti disponibili nella classe CdL
- per realizzare tale servizio, un oggetto CdL dovrà chiedere singolarmente ad ognuno degli oggetti studente di sapere se è stato superato Analisi I, e con che voto.
- Ogni studente chiede ai propri oggetti Esame se esiste l'oggetto con attributo Analisi I, e che voto questo eventualmente ha.
- Alla fine, l'oggetto CdL raggruppa i dati forniti dagli oggetti studenti per restituire il numero totale di esami Analisi I superati.

SEGUE ESEMPIO:

Alcune Note Importanti (considerazioni generali su Programmi ad Oggetti)

1. gli oggetti vengono creati quando servono, non si ha semplicemente una dichiarazione iniziale di tutti gli oggetti (come succede per le variabili in programmi C)

2. gli oggetti devono in qualche modo "conoscersi" l'un l'altro per poter interagire, richiedendosi servizi l'un l'altro

3. tale conoscenza si ha attraverso "riferimenti" tra oggetti, che rappresentano in qualche modo degli "indirizzi": un oggetto può conoscere altri oggetti perché, tra le sue variabili o attributi, ha dei valori che rappresentano indirizzi di altri oggetti, e può usare tali indirizzi per mandare messagi finalizzati al chiedere "servizi" agli oggetti che conosce

4. Un oggetto che crea un altro oggetto, necessariamente conosce il suo indirizzo (una mamma sa che sua figlia è lì con lei…)

5. Un oggetto che non ha creato un altro oggetto ma ha bisogno di chiedergli servizi deve in qualche modo venire a conoscenza del suo indirizzo, che può essere comunicato dal creatore ad altri oggetti tramite una sorta di "passaparola" (ci si scambia gli indirizzi tra oggetti…). PER ESEMPIO: un oggetto studente non ha creato l'oggetto Conta Esami CdL, che è stato creato dall'oggetto CdL stesso; però, se quando l'oggetto CdL crea un nuovo studente gli comunica (cioè gli passa come parametro) l'indirizzo dell'oggetto Conta Esami CdL; così quando uno studente supera un nuovo esami,

VANTAGGI DELLA PROGRAMMAZIONE AD OGGETTI

Facilitazione di costruzione *cooperativa* di software:

- diverse persone sviluppano diverse classi
- ogni programmatore può semplicemente verificare il comportamento delle sue classi istanziandone oggetti e verificandone il comportamento in risposta a richieste di servizio
- unico accordo necessario per integrare il tutto in un unico sistema finale: definire le interfacce (quali metodi e quali parametri) delle classi

Facilitazione della *gestione e manutenzione*

- Se vi sono errori sui dati in un oggetto, è facile scoprire dove si trova l'errore, perché (siccome i dati non sono visibili all'esterno dell'oggetto) esso non potrà essere che all'interno dell'oggetto che gestisce quei dati
- le modifiche a una classe non rendono necessario modificare il resto del programma (le altre classi) a meno che non venga modificata l'interfaccia

Supporto a progettazione e sviluppo *incrementali*

- si possono definire nuove classi sfruttando il codice di classi già esistenti (p.e., una classe studente a partire da una classe persona)

Possibilità di *rapida prototipazione*

- non importa che una classe sia completamente definita per poter iniziare a "testare" il funzionamente degli oggetti di quella classe.